

Creating and Sharing Genomic Annotation Modules with AI: The Just-DNA Ecosystem

Anonymous submission to EASRP 2026

Abstract

Genome annotation depends on curated knowledge that changes as papers and databases publish new variant-trait associations. The Just-DNA ecosystem stores this knowledge in genomic annotation modules: versioned data packages that describe which genotype a claim concerns, what evidence supports it, and how a consumer should present it. A module can be written and edited manually. An AI assistant can also draft one from a paper, a variant list, or a description of a topic, but its output remains an editable data artifact rather than generated analysis code.

We present the Just-DNA ecosystem and the path a module follows through it. `just-dna-format` defines the schema; `just-dna-enricher` records source-derived facts; `just-dna-compiler` validates and compiles the files. `just-module-creator`, an open-source plugin for Claude Code and Codex, is the authoring entry point: it exposes these packages as typed tools so that both AI-assisted and manually edited modules use the same live schema. An author can keep the result in a local module store, rehearse publication in a deletable test registry, or publish a version to the production registry. Just-DNA-Lite later filters a local VCF and joins selected modules to matching genotypes. Polygenic scores follow a parallel numerical path through `just-prs` (described in a companion platform paper).

The language model helps with reading and drafting, while deterministic software handles schema validation, compilation, genome processing, and scoring. This paper describes the module lifecycle from source material to shared store and local use. It also defines a protocol for evaluating variant recovery, citation identity, and effect-direction agreement in AI-assisted creation. The software is open-source and intended for research use only.

1 Introduction

Suppose a researcher wants a recent lactose-tolerance study to inform a genomic report. The researcher can transcribe the relevant variants and evidence into tables, or ask an AI assistant to prepare a first draft. In either case, the result is the same kind of object: a module that can be edited, checked, versioned, and shared. The researcher may keep it in a local store while it is being reviewed, publish it to a test registry for rehearsal, and later release an accepted version through the production registry. When a user selects that module in Just-DNA-Lite, the application joins its records to matching genotypes in a local VCF and includes the resulting annotations in a report.

This separation between knowledge and sample processing is central to the design. A paper may report an association with lactose tolerance, drug response, or another trait, but a computational pipeline needs an unambiguous variant, the genotype to which the claim applies, the genome build and allele orientation, a structured conclusion, and a trace back to the evidence. A *module* is a versioned collection of those records. It contains no sample data and no executable analysis code. The same compiled module can be applied to many genomes.

The sample follows a different path. A VCF often contains millions of variant calls. Just-DNA-Lite normalizes their representation, applies configured quality filters, and joins the retained calls to selected modules and reference data. Polygenic risk scores follow a parallel numerical path: `just-prs` applies a PGS Catalog model to the sample and places the score in the context of a reference population. These maintained pipelines can reuse normalized columnar data instead of reparsing the original VCF for every question.

A general-purpose assistant is useful during curation, but it does not provide this structure by itself. Asked to “make me a lactose-tolerance module,” it may return rsIDs, an explanation, and

a new script that searches a VCF. The answer can contain a fabricated identifier, a reversed allele, or a real PMID that names the wrong paper [Ji et al., 2023, Gao et al., 2023]. Generated code may also bypass the established filters and validators or depend on an unstated environment. The next run can produce a different implementation because the implementation was generated as part of the answer.

The Just-DNA ecosystem gives that assistant a narrower and more useful job. `just-module-creator` helps it read sources, prepare module files, run checks, and present unresolved decisions to the author. The files use schemas from `just-dna-format`; `just-dna-enricher` records facts from external sources; `just-dna-compiler` validates and compiles them; and `just-dna-registry` stores published versions. Manual authors use the same files and tools. Just-DNA-Lite and `just-prs` remain responsible for sample-level computation.

This paper follows a module through the ecosystem, from manual or AI-assisted authoring to a shared store and local use in a genomic report. Just-DNA-Lite and `just-prs` are described as the consumer side of the boundary; their internal design is the subject of a companion platform paper.

This paper makes four contributions:

1. It describes the module contract shared by manual editors, AI-assisted authoring tools, versioned stores, and the genome-analysis consumer.
2. It presents `just-module-creator`, an authoring plugin that exposes the public capabilities of `just-dna-format`, `just-dna-compiler`, `just-dna-enricher`, and `just-dna-registry` as typed MCP tools. The plugin reads the installed schema instead of carrying a copy.
3. It defines how modules move between a local store, a deletable test registry, and the production registry while preserving provenance and decisions that still require review.
4. It defines an evaluation protocol for variant recovery, citation identity, and effect-direction agreement. Compiler round-trip behaviour is tested by `just-dna-compiler` and is not counted again as module creation accuracy.

2 Related work

Genome annotation and catalogs. A variant call records what differs between a sample and a reference genome. Annotation adds information about that call, such as its predicted consequence, known clinical classification, reported trait association, or the conclusion assigned to a particular genotype. VEP, ANNOVAR, and OpenCRAVAT/OakVar perform this kind of lookup by joining a callset to established resources [McLaren et al., 2016, Wang et al., 2010, Pagel et al., 2020]. ClinVar and the GWAS Catalog publish records that can supply annotation content [Landrum et al., 2018, Sollis et al., 2023]. The problem addressed here begins before the join: selected evidence must be turned into a reviewable, machine-checkable collection that can be distributed and applied consistently.

Language models and scientific extraction. Language models can produce fluent text around a false identifier or citation [Ji et al., 2023]. Tool use reduces the need to improvise code when a model can call a typed operation instead [Schick et al., 2023, Anthropic, 2024]. Biomedical MCP services can search literature, ontologies, and variant databases. `just-module-creator` uses the same general approach, then carries the result into module files that the rest of the Just-DNA pipeline can inspect and reuse.

Agent orchestration. Many scientific-agent systems assign research and review to several model instances. The plugin described here does not require its own orchestration runtime. Claude Code or Codex supplies the agent, while the plugin supplies the tools and the written workflow. This paper evaluates that authoring path, not a particular choice of language model or agent-team topology.

3 Method

3.1 How modules move through the ecosystem

Module authoring and sample analysis remain separate until Just-DNA-Lite joins a compiled module to a local genome. On the authoring path, a person may edit the files directly or use an AI assistant to draft them. Both routes then pass through the same enrichment and compilation steps before the module enters a local or shared store.

Table 1: The module lifecycle and the two sample-processing paths.

Path	Stages
Module lifecycle	papers and databases → manual editing or AI draft → enrichment and compilation → local store or registry
Sample annotation	VCF → normalization and quality filtering → joins with selected modules and reference data → annotated tables and report
Polygenic scoring	VCF plus a PGS Catalog model → just-prs computation → score and reference-population context → report

On the sample path, normalization gives equivalent variants a consistent representation and computes the genotype used by later joins. Configured quality filters can remove calls that fail VCF filter status, depth, or quality criteria. The pipeline writes the normalized data to Parquet, so each selected module can use a streaming join instead of independently parsing the original VCF. An optional Ensembl join adds reference annotations. Report generation reads the resulting module-specific Parquet files and their display metadata.

Polygenic risk scores are part of the ecosystem but not calculations performed by **just-module-creator**. Just-DNA-Lite delegates them to **just-prs**, which applies published PGS Catalog scoring models to the sample. A module can describe a published score in **pgs.csv**; the numerical score still belongs to **just-prs**. This keeps literature curation and sample-level computation separate while allowing both results to appear in the same application.

Within **just-module-creator**, the language model participates only in the knowledge path. It can search and read sources, draft module rows, and help review disagreements. An assistant may launch Just-DNA-Lite or **just-prs** through their established interfaces, but it does not replace their implementations. VCF filtering, module joins, and polygenic scoring remain ordinary software steps with explicit inputs and repeatable code.

3.2 The module artifact

A just-dna module begins as a directory. It always contains **module_spec.yaml** and at least one recognized table. The table records the thing being annotated, such as a genotype, a diplotype, or a measured quantity. Evidence and machine-produced sidecars are stored separately. The compiler turns this directory into parquet files and a content-addressed **manifest.json** [DNA Seq contributors, 2026].

The authored files are ordinary YAML, CSV, Markdown, and optional image files. They can be created and revised without an AI assistant. **just-module-creator** operates on this same directory rather than introducing an AI-specific module format. The module describes how a

genotype or measurement should be interpreted if a consumer encounters it. Just-DNA-Lite, or another compatible consumer, supplies the genome or measurement later.

The table kind follows the subject of the claim. For example, in format 0.6.6 used for this manuscript, a single-variant claim belongs in `variants.csv`; evidence for those rows belongs in `studies.csv`. Pharmacogenomic diplotypes, copy-number ranges, repeat counts, and published polygenic-score identifiers use other tables. The manuscript does not reproduce their column lists. The plugin calls `describe_table` and `table_requirements`, which read the installed pydantic models. An author therefore sees the schema accepted by the compiler running in that session.

3.3 Components and responsibilities

No single package implements the whole path. Each component owns a boundary that another component can call without reimplementing its rules. Within the module toolchain, dependencies point inward: `enricher` $\rightarrow$ `compiler` $\rightarrow$ `format`. The format and compiler do not fetch remote data. The enricher performs source lookups and writes the derived sidecars used later by the compiler. The registry publishes the compiled result, and Just-DNA-Lite consumes it in a separate application. Polygenic scoring is delegated to `just-prs`.

Table 2: Responsibilities across the Just-DNA ecosystem.

Component	Responsibility	Does not
<code>just-module-creator</code>	research and authoring workflow	process a genome
<code>just-dna-format</code>	schema and identity rules	access the network
<code>just-dna-compiler</code>	validate, compile, reverse, and sign	access the network
<code>just-dna-enricher</code>	resolve, draft, and cross-check	decide scientific meaning
<code>just-dna-registry</code>	publish, discover, and download modules	author module rows
Just-DNA-Lite	filter VCFs, join annotations, and produce reports	define module schema or compiler rules
<code>just-prs</code>	compute polygenic scores	create annotation modules

Of these components, `just-module-creator` is the agent-facing authoring entry point examined most closely in this paper. It is distributed as an application whose public contract is the MCP tool surface and the accompanying workflow documents. The same source tree ships plugin manifests for Claude Code and Codex. Both hosts launch the same server and load the same skills.

3.4 Tools available to the assistant

The MCP tools are grouped by task: authoring, research, checks, enrichment, comparison, and registry work. The default configuration lists the full surface. An optional layered configuration initially lists the core authoring loop and a `toolbox` command that reveals the other groups. Tool search can reduce the listing further. These discovery options change what the agent sees in its context; they do not create a separate implementation of the workflow.

Registry writes require a token, with one necessary exception: `registry_register` creates the token and therefore cannot require one in advance. Tools whose cost depends on a large source corpus state that cost in their descriptions.

Literature and enrichment requests share a pacing gate, including the NCBI budget used by several lookups. The literature search fills a gap left by the enricher. The enricher can verify a PMID that an author already has, but it does not search for the paper. `literature_search`

returns titles so the assistant can check identity. The existence of a PMID alone says nothing about whether it is the intended article.

3.5 How a person enters the workflow

A person usually knows what they want to accomplish, not which internal stage performs it. They may want to create a module, understand an inherited directory, find a paper, revise a published module, or decode an error message. The command menu uses those intentions as its entries.

Command	Purpose
<code>/create-module</code>	begin or continue creating a module
<code>/module-status</code>	inspect a directory and identify the next decision
<code>/module-revise</code>	begin another pass over an existing module
<code>/find-evidence</code>	find and read evidence for a row
<code>/module-publish</code>	rehearse and then publish
<code>/module-symptom</code>	explain a message from the toolchain
<code>/module-install-local</code>	install a compiled module without a catalog

The first version of the menu exposed twenty stage names. That forced a person to choose between terms such as `module-curate` and `module-enrich` before the system had explained either one. The seven commands now route to thirteen guides. A guide contains the procedure for one stage and is loaded by path when the assistant reaches that stage.

`/create-module` is therefore a router. It examines what the author has already provided and selects the next stage. If the request shows that the person first needs an explanation, the router loads the overview guide. The overview is not a menu command because a new author would not know its internal name.

3.6 The authoring journey

The workflow follows the module from an idea to one of its destinations:

`origin` → `scaffold` → `draft` → `curate` → `enrich` → `check` → `compile` → `close` →
`rehearse` → `publish`

The origin determines where the work begins. A source such as ClinVar, CPIC, or ClinPGx may already publish rows, in which case a drafting tool can prepare a partial table. A paper or a free-text request instead begins with evidence search and manual authoring. Scaffolding creates the files and placeholders required for the selected table kind.

Curation follows drafting because a drafted row may still contain decisions that the source cannot make for the module. The author or assistant must decide which genotype the claim concerns, how a weight should be interpreted, and how the conclusion should be worded. The workflow presents these decisions rather than silently choosing values that merely satisfy the schema.

Enrichment resolves identifiers and writes sidecars such as `resolution.csv` and `literature.csv`. These files record the source answers used for later offline compilation. Compilation itself does not use the network. The plugin always calls the compiler with `resolve_with_ensembl=True`. The parameter name is misleading: setting it to false disables all resolution, including an injected `resolution.csv`, and the compiler can then succeed with null coordinates that cannot match a VCF.

After compilation, `close` records a statement in `verification.json` and binds it to the authored files. Editing those files invalidates the closure. In the current 0.x format a module can still compile and publish without a closure, but it carries a warning. Closing is an attributed declaration of completion. It does not prove that the module is correct.

Most modules return to this workflow. A later source release, a new paper, or a reviewer's correction begins a revision pass. The router inspects the existing directory so that the assistant does not treat a revision as a blank first draft and overwrite decisions whose history matters.

3.7 Edits, evidence, and responsibility

The compiler reports problems but does not edit authored values. `just-module-creator` is the authoring application, so its workflow permits edits. That permission comes with limits.

First, tool-mediated overrides append a record to `logs/authoring.log` and preserve a reason in `provenance.json`. A general hand edit is not captured automatically. The server therefore instructs an assistant to call `record_override` after such a change. This is a known gap between the desired provenance record and what the current tools can enforce.

Second, the workflow does not fill a value from the source that will later be used to check that same value. Such a check would compare the source with itself. The same problem appears when a row uses an article title as its `provenance_quote`: the title is guaranteed to occur in the article, so the resulting match provides no evidence that anyone located support for the row's claim.

Third, disagreement with an archive requires review of both sides. An archive may lag a retraction, a meta-analysis, or a later reclassification. Replacing the module's value with the archive's value can therefore make a module worse. `record_override` stores which field differs, what the source said, who made the decision, and why the authored value should remain. The record does not turn the disagreement into a passed check.

An assistant may read retrieved full text and locate a `provenance_quote`. It must copy the passage verbatim and identify who located it in `StudyRow.curator`. This attribution helps a reviewer decide where to look closely. It does not transfer responsibility away from the human author.

3.8 Local and shared module stores

In this paper, a module store is a place from which a compiled module can be installed or discovered. A local installation is useful during authoring. The registry provides two shared stores for publication rehearsal and release.

The registry has two independent instances. Production is the catalog used by consumers. The polygon, selected with `target=test`, is a rehearsal environment where an author can delete a test publication. Writes default to the polygon. Catalog reads require an explicit target so that an author does not publish to one instance, read from the other, and mistake the missing result for a failed publication.

The production catalog is immutable. Publishing a version also claims its authored content by hash, and yanking the version does not release that claim. The workflow therefore rehearses on the polygon and asks separately before a production publish. The registry accepts the specification, runs its own enrichment and strict compilation, and stores the artifact it produced. It does not rely on a locally claimed digest, and it does not require the author to sign the local artifact.

Publication is not the only way to use a module. `/module-install-local` installs a compiled artifact into a Just-DNA-Lite checkout so it can be tested with a local VCF. This route tests the annotation connection between the module and the consumer. A polygon publication tests the separate connection between the module and the registry.

4 Results

4.1 Implemented surface

`just-module-creator` is released in the 0.24 series at the time of this draft. The same source tree loads in Claude Code and Codex. After `uv sync`, the tools named by the authoring workflow are registered. The former essentials and extended modes are gone; the optional layered listing only changes how tools are revealed to the agent.

Schema answers in the environment used for this manuscript come from `just-dna-format` and `just-dna-compiler` 0.6.6. The plugin does not maintain a separate list of columns or vocabularies.

The command redesign reduced the text placed in every session from 14,688 characters for twenty entries to 3,464 characters for seven entries, a 76% reduction. A test walks the links from those seven commands and confirms that every guide remains reachable. The smaller menu therefore removes repeated prompt content without making a stage inaccessible.

4.2 Compiler behaviour belongs to the compiler

Within one compiler version, `just-dna-compiler` tests that a valid specification can be compiled, reversed, and compiled again without changing its authored content signature. `just-module-creator` calls that compiler and fixes the resolution setting described above. It does not implement a second compiler or present the upstream round-trip tests as evidence that an assistant extracted a paper correctly.

This separation avoids a circular evaluation. If an assistant writes a wrong claim in valid syntax, the compiler may preserve it perfectly. Reproducible output shows that the compiler kept the input stable; it does not show that the input agrees with the literature.

4.3 Protocol for creation accuracy

The creation evaluation begins with an expert-checked fixture. Each fixture contains a free-text prompt and the `module_spec.yaml`, `variants.csv`, and `studies.csv` that the assistant should recover. The generated module is compared with the fixture on three questions:

- Variant recall is the fraction of ground-truth (*rsID*, *genotype*) rows present in the generated module.
- Citation identity requires every cited PMID to exist and to name the paper indicated by the title returned during search. Existence alone is insufficient.
- Weight-sign accuracy measures whether the generated effect direction agrees with the fixture. Magnitude is reported separately because a module weight is an authored choice and is not copied from a GWAS beta.

The development fixtures `fto_bmi`, which contains one locus at rs1421085, and `longevity_2026` are too small to support a performance claim. A larger adjudicated set and repeated runs are still needed. We therefore report no recall or precision estimate in this draft.

4.4 How to read a green result

Several outputs look reassuring while answering a narrower question than a reader may expect. Strict compilation checks reproducibility and applies the compiler's blocking rules; it does not establish biological correctness. A digest match likewise shows that bytes or authored content agree under a specified comparison.

A source timeout produces an unknown result, not a negative one and not a pass. The tools preserve this distinction with null and unknown values. The quote example is another useful test of a green check. If every row cites its article title as the supporting passage, the full-text matcher will find every quote even though nobody located evidence for any individual claim.

The creation protocol counts none of these as evidence of accurate extraction. It remains a protocol until it has been run on a sufficiently large fixture set using this plugin.

5 Discussion

The ecosystem’s architecture changes what an AI assistant is asked to produce. It writes module data for an existing pipeline instead of inventing the pipeline again for each question. The same compiler and schema are used for every module, and the sample is normalized once before selected annotations are joined through the established columnar implementation. The language model may still misunderstand a paper. Its draft is kept inspectable, and source disagreements and unresolved judgements remain visible to the author.

The workflow must work for people with different levels of expertise. A geneticist may want to decide every genotype and evidence statement. A non-specialist may provide a topic and several papers, then rely on the assistant to explain each decision. The tool surface cannot assume which kind of author is present. It can perform mechanical corrections and keep a record of them, while presenting scientific interpretation and production publishing for review.

The skills help the agent follow this path, but they are instructions rather than a security boundary. An agent can still bypass them and edit a CSV directly. The same is true of the provenance log: tool-mediated overrides are recorded, while a hand edit depends on the agent calling the recording tool. These limits are important because the paper evaluates a workflow composed of tools and instructions, not a closed application that prevents every other action.

Agent hosts and MCP conventions change quickly. This paper focuses on the parts that should survive a client update. Schema answers come from the installed models. Edits can carry attribution, and source disagreements remain visible. Publication is rehearsed on a separate instance. The compiler retains authority over the artifact. Client-specific setup belongs in the repository documentation.

The present evaluation has several limitations. It contains no creation accuracy estimate, and the available fixtures cover too few kinds of module. The plugin cannot determine whether an annotation is medically correct. It can make the claim structured, attributable, and easier to review. The module authoring workflow targets GRCh38 and does not perform liftover. The wider ecosystem does compute polygenic scores through `just-prs`, but that numeric path is not evaluated in this paper. Local installation into Just-DNA-Lite is described as a procedure that runs in that checkout; `just-module-creator` does not execute the consumer on the author’s behalf. We also have not compared the runtime of freely generated VCF scripts with the Just-DNA-Lite pipeline. The performance argument made here is architectural: the established path reuses normalized Parquet data and a maintained join implementation instead of generating a new analysis for each request.

6 Conclusion

The Just-DNA ecosystem separates two kinds of work. Module authors turn papers and databases into structured, attributable annotation knowledge. Just-DNA-Lite filters and normalizes a local genome, joins it to selected compiled modules, computes polygenic scores through `just-prs`, and produces results for inspection and reporting. A companion platform paper describes that consumer side in detail.

Within the authoring side, `just-module-creator` lets an AI assistant draft and revise ordinary module files while the installed schema, enricher, compiler, and registry provide the shared path

from draft to published artifact. A module can then be installed locally or consumed from the catalog without asking the language model to write another genomic pipeline. Scientific judgement belongs to the author, and sample-level computation belongs to deterministic software. The software is open-source and intended for research use only.

References

- Anthropic. Model context protocol. <https://modelcontextprotocol.io>, 2024. Specification and documentation.
- DNA Seq contributors. just-dna-format: schema, compiler, and enricher for just-dna annotation modules. <https://github.com/dna-seq/just-dna-format>, 2026. Software. Packages just-dna-format, just-dna-compiler, just-dna-enricher.
- Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. *arXiv preprint arXiv:2305.14627*, 2023. URL <https://arxiv.org/abs/2305.14627>.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730.
- Melissa J. Landrum, Jennifer M. Lee, Mark Benson, Garth R. Brown, Chen Chao, Shanmuga Chitipiralla, Baoshan Gu, Jennifer Hart, Douglas Hoffman, Wonhee Jang, Karen Karapetyan, Kenneth Katz, Chunlei Liu, Zenith Maddipatla, Adriana Malheiro, Kurt McDaniel, Michael Ovetsky, George Riley, George Zhou, J. Bradley Holmes, Brandi L. Kattman, and Donna R. Maglott. ClinVar: improving access to variant interpretations and supporting evidence. *Nucleic Acids Research*, 46(D1):D1062–D1067, 2018. doi: 10.1093/nar/gkx1153.
- William McLaren, Laurent Gil, Sarah E. Hunt, Harpreet Singh Riat, Graham R. S. Ritchie, Anja Thormann, Paul Flicek, and Fiona Cunningham. The Ensembl variant effect predictor. *Genome Biology*, 17:122, 2016. doi: 10.1186/s13059-016-0974-4.
- Kymerleigh A. Pagel, Rick Kim, Kyle Moad, Ben Busby, Lily Zheng, Collin Tokheim, Michael Ryan, and Rachel Karchin. Integrated Informatics Analysis of Cancer-Related Variants. *JCO Clinical Cancer Informatics*, 4:310–317, 2020. doi: 10.1200/CCI.19.00132. OpenCRAVAT. OakVar is the maintained successor.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Elliot Sollis, Annalisa Mosaku, Ala Abid, Annalisa Buniello, Maria Cerezo, Laurent Gil, Tudor Groza, Osman Güneş, Peggy Hall, James Hayhurst, Arwa Ibrahim, Yue Ji, Sajo John, Elizabeth John, G. Naamati, Michael Nicholson, A. Oyewole, E. Palumbo, Nigel W. Rayner, C. Sacramento, S. Trevanion, M. Zaharia, A. McMahon, L. Harris, H. Parkinson, and F. Cunningham. The NHGRI-EBI GWAS catalog: knowledgebase and deposition resource. *Nucleic Acids Research*, 51(D1):D977–D985, 2023. doi: 10.1093/nar/gkac1010.
- Kai Wang, Mingyao Li, and Hakon Hakonarson. ANNOVAR: functional annotation of genetic variants from high-throughput sequencing data. *Nucleic Acids Research*, 38(16):e164, 2010. doi: 10.1093/nar/gkq603.

A Code and data availability

- **just-module-creator**: <https://github.com/dna-seq/just-module-creator> (MCP server, Claude Code and Codex plugin manifests, skills; MIT).
- **just-dna-format, just-dna-compiler, and just-dna-enricher**: <https://github.com/dna-seq/just-dna-format> (schema, reference compiler, and enricher).
- **just-dna-registry**: production catalog at <https://module-registry.just-dna.life> and polygon at <https://module-polygon.just-dna.life>.
- **Just-DNA-Lite**: <https://github.com/dna-seq/just-dna-lite> (local VCF processing, annotation, and reporting; described in a companion platform paper).
- **just-prs**: <https://github.com/dna-seq/just-prs> (polygenic risk-score computation library).

B Research use only

Annotation modules summarize published association findings. They are not clinical-grade evidence. The authoring plugin does not make individual-level predictions and does not open a genome. Just-DNA-Lite and **just-prs** process sample data for research and educational use; their reports and scores are not diagnoses or medical advice. Language that implies causation, such as “causes” or “guarantees,” is outside the scope of a module written with this plugin.